

Billy Bones Inn / Ren'Py Project Architecture Knowledge Base

1. Main Development Rule

This project must be developed as a clean, template-like Ren'Py game engine.

The goal is not only to make one scene work, but to keep the project expandable:

- new locations
- new rooms
- new NPCs
- new items
- new events
- new story threads
- new systems such as fighting, mini-games, travel, jobs, economy, hunting, fishing, shop systems
- new debug/test screens

Every new feature must follow the existing architecture and must not create redundant parallel systems.

The most important rule:

One source of truth per system.

Do not duplicate the same data or behavior in objects, labels, screens, and helper functions at the same time.

2. Source-of-Truth Rules

Rooms

Room objects are the source of truth for room data.

Room objects own:

- `code_name`
- `display_name`
- `group_name`
- background key / picture
- descriptions
- exits
- game objects
- room actions
- visibility conditions

Location labels are only Ren'Py flow anchors.

A location label may:

- set `CurLoc`
- set `location = CurLoc`
- set `CurrentRoom`
- call room-enter event gate
- call the room template / main UI

A location label must not duplicate room descriptions, exits, item lists, or menus if those already exist in the Room object.

Items

`GameItem` objects are the source of truth for item data.

A game item owns:

- `object_id`
- `display_name`
- description
- category
- price/value
- item-specific data
- possible actions

Catalogs and category lists organize items.

Inventories store item IDs, not full item objects.

Example:

```
GameItem object: WorkDressItem
item ID: dress_workdress
dress code: workdress
display name: Бежевое повседневное платье
```

Do not confuse item ID with clothing/dress code.

Clothes / Dress System

The clothing system has two layers:

```
Item layer:
GameItem, item catalog, menDress, womenDress, shop lists
```

```
Dress-code layer:
DressDesc, FullDressDesc, ShortDressName, DressTopPart, DressBottomPart,
DressPartDesc, DressPartSlut, MaleDressCodes, FemaleDressCodes,
MyDresses, MyCurDress, dressdefault, topdress, bottomdress
```

The tailor shop must use the catalog/category lists to display male and female clothing.

Dress application logic must use dress codes.

Events and Threads

Event/thread objects are the source of truth for:

- availability
- conditions
- trigger location
- trigger action
- priority
- stage
- active/pending/completed/blocked status
- repeat rules
- what label should run
- what thread advances next

Event labels are the source of truth for authored scene presentation:

- text
- dialogue
- image/vscene calls
- menus
- player choices
- immediate result text
- scene-specific variable updates

This is not duplication. This is correct separation.

Do not duplicate event availability conditions inside event labels if they are already owned by the event object/manager.

3. Naming Contract

Avoid ambiguous words like `name` unless old code already uses them.

Use this distinction everywhere:

Object variable:

The Python/Ren'Py variable holding the object.

Example: TavernKitchenRoom, AmandaNPC, WorkDressItem

code_name / object_id / event_id:

Stable internal string key used by logic, saves, registries, navigation, and triggers.

Example: "TavernKitchen", "amanda", "dress_workdress", "amanda_breakfast_1"

display_name:

Player-facing visible text.

Example: "Кухня", "Аманда", "Бежевое повседневное платье"

Global state should store IDs/strings, not full objects.

```
default CurLoc = "TavernMain"
default location = CurLoc
default ActiveEvent = ""
default CurrentNPC = ""
```

`CurLoc` is always the current room/location code string.

`location` is a legacy mirror of `CurLoc`.

`CurrentRoom` is derived from `CurLoc`.

Do not assign a Room object to `CurLoc`.

4. Dicts vs OOP

Before adding or converting a system, decide whether it uses dictionaries or OOP.

Do not mix both styles randomly.

Use OOP when:

- the object has behavior/methods
- the object appears in many places
- the object has visibility conditions
- the object has actions
- the object is part of a reusable engine system

Examples:

- Room
- RoomExit
- RoomDescription

- GameItem
- GameObject
- StoryEvent
- Thread
- PeopleData
- NPC runtime objects

Use dictionaries when:

- the data is simple save-state
- the structure is legacy and already stable
- the values are story flags or relationship stats
- the data is mostly key/value runtime state

Examples:

- AmandaVar
- MelissaVar
- SandraVar
- Friends
- CurrentLoc
- dressdefault
- topdress
- bottomdress

Do not convert legacy dictionaries into classes unless the conversion plan is explicit.

5. Label vs Method Responsibility

Methods/functions perform reusable logic.

Labels control Ren'Py scene flow and authored presentation.

Do not implement the same action twice.

Bad:

A Python method changes state.
A label also changes the same state independently.

Good:

Method/function:
checks conditions, updates variables, returns result

Label:
calls method, shows text/image/menu, returns to caller

Example:

```
label ChopWood:
    $ result = chop_wood()

    if result == "success":
        $ MainTxt = "Вы нарубили дров."
    else:
        $ MainTxt = "Вы слишком устали."

    call screen main_ui
    return
```

6. Triggered Event Flow Contract

Events called through `RoomEnterEventGate` / `checkTriggers` must return to the caller.

They must not jump back to the room.

Forbidden inside triggered events:

```
jump expression CurLoc
Jump(CurLoc)
jump StreetTavern
call StreetTavern
```

Allowed:

```
return True
Return(True)
return False
```

Correct flow:

```
Room label
→ RoomEnterEventGate
→ checkTriggers
→ event label
→ return True
→ checkTriggers returns
→ RoomEnterEventGate returns
→ room label rebuilds room UI
→ main_ui
```

If an event label is standalone and entered by `jump`, then `jump expression CurLoc` may be acceptable. But if it is called by the room-enter pipeline, it must return.

7. UI State Reset Rule

Event labels can temporarily own:

- `MainTxt`
- `CurLocDesc`
- `current_action_title`
- `current_action_content`
- `current_action_items`

After a triggered event returns, the room must rebuild its own menu from scratch.

Before building room navigation, reset:

```
$ current_action_title = "Куда идти"
$ current_action_content = None
$ current_action_items = []
$ current_girl_key = ""
$ current_object_id = ""
```

This prevents event menus from sticking together with normal navigation menus.

8. Calendar Contract

The calendar must have one canonical mutable source of time.

Recommended canonical state:

```
default dayspassed = 0
default clock_minutes = 480
```

Everything else should be derived:

```
hour = clock_minutes // 60
minute = clock_minutes % 60
time = derived time-of-day slot
weekday = dayspassed % 7
week = dayspassed // 7 + 1
month/period = derived from dayspassed
cycle = derived from dayspassed
```

Labels must not manually update hour, minute, week, month, cycle, and time slot separately.

Use one time advancement function:

```
$ advance_minutes(30)
```

Daily resets must happen only inside the calendar/new-day system.

9. HUD / Screens

Screens display prepared state.

Screens do not own game logic.

HUD may display:

- current location
- current time/calendar
- player stats
- chores
- NPCs present
- current text
- current action buttons
- inventory summaries

HUD must not decide:

- whether an event triggers
- whether an NPC moves
- whether a room changes
- whether an item is bought
- whether a quest advances

Those decisions belong to managers, functions, labels, or object methods.

10. QSP Legacy Conversion Rule

When converting legacy QSP content, do not immediately write Ren'Py architecture.

First step:

```
Sort QSP locations by functionality.
```


Classify each QSP location/text file as one of:

- physical location / room
- NPC interaction
- event scene
- thread/story progression
- shop
- mini-game
- combat/fight logic
- inventory/item logic
- utility function
- calendar/time logic
- image/media display
- debug/test logic
- obsolete/dead code

Second step:

Decide whether this system should remain dict-based or become OOP-based.

Use the project architecture:

- rooms become Room objects + location labels
- items become GameItem objects + catalog/category lists
- events become StoryEvent/thread definitions + Ren'Py labels
- NPC identity becomes PeopleData
- NPC state may remain dictionaries if already stable
- reusable logic becomes functions/methods
- authored scenes remain labels

Third step:

Convert using Ren'Py programming culture.

That means:

- labels for authored scene flow
- screens for display
- Python functions/classes for reusable logic
- `default` for save-state variables
- `define` for static definitions
- no hidden adapters unless necessary
- no duplicate object/label ownership

11. Debug and Human Visual Testing

Every new feature must be testable without playing the whole game.

Add debug/test support for:

- entering a specific location
- forcing a specific event
- testing a room's visible exits/actions
- testing NPC presence
- testing item inventory/shop lists
- testing clothing/dress lists
- testing calendar/time states
- testing fight/mini-game systems
- testing newly added story threads

Debug screens are for human visual testing.

Example use cases:

Added new location:

Open debug location list → jump to location → verify image/text/exits/actions.

Added event to location:

Open debug event screen → force event → verify text/menu/results/return path.

Added dress item:

Open debug shop/inventory screen → verify male/female dress list → buy/wear/test.

Changed calendar:

Open debug time screen → advance time/day/week → verify HUD and triggers.

Debug systems must not become gameplay systems.

They are tools for testing specific changed features.

12. New Feature Rule

Before adding any new feature, answer:

What object owns the data?
What function/method owns the logic?
What label owns the scene flow?
What screen displays it?
What state variables are changed?
How is it tested directly?

If the answer is unclear, do not code yet.

13. Agent Behavior Rules

The agent must not invent architecture.

The agent must follow existing project contracts.

The agent must avoid:

- new redundant wrappers
- new aliases for existing state
- converting objects into dicts
- converting dicts into objects without explicit plan
- bypassing existing systems unless explicitly requested
- changing unrelated files
- “fixing” by adding another runtime layer
- using `jump CurLoc` inside called trigger events
- duplicating room data inside labels
- hiding simple Boolean flags behind pointless wrapper functions

The agent should prefer:

- minimal diffs
- exact file paths
- exact replacement blocks
- preserving current naming when it is already established
- comments explaining why, not obvious what
- local debug/test entry points
- restoring broken files from Git when needed

14. Building Blocks Summary

The project is built from these blocks:

```
GameItem:
item definitions, object IDs, categories, item data

Item catalog/category lists:
lookup and shop/inventory grouping

Room:
location data, descriptions, exits, objects, room actions

RoomTemplate/main_ui:
renders current room state

StoryEvent / Thread:
conditions, state, priority, progression
```

Event labels:
authored scenes, images, text, menus, consequences

PeopleData:
character identity and static/semi-static info

NPC runtime:
current behavior, enemy/neutral/ally/service role, combat/interaction state

Calendar:
canonical time progression and derived display values

HUD:
visual display of current state and action buttons

Debug/test screens:
human-controlled feature testing

15. Final Principle

The project must stay editable.

A human should be able to open a file and understand:

what this file owns
what calls it
what it calls
what state it changes
how to test it

If a file cannot answer those questions, the structure is too muddy and must be simplified.

Final rule:

Objects define what exists.
State variables store current IDs and progress.
Managers decide what is available.
Methods perform reusable logic.
Labels present authored Ren'Py scenes and control flow.
Screens display the prepared state.
Debug tools test isolated features.